

3.5.3 SLR(1)分析表的构造

LR(0)文法是一类非常简单的文法，其特点是该文法的活前缀识别自动机的每一状态(项目集)都不含冲突性的项目。但是，即使是定义算术表达式这样的简单文法也不是LR(0)的，因此，需要研究一种简单“展望”材料的LR分析法，即SLR(1)法。

实际上，许多冲突性的动作都可以通过考察有关非终结符的FOLLOW集(即紧跟在该非终结符之后的终结符或“#”)而获得解决。

4.6.4、SLR(1) 技术

LR(0) 方法由于只看状态栈的内容，而不看输入符，因此容易出现冲突状态(如表达式文法)。如果不仅看状态栈的内容，还看当前输入符，则因为多用了信息自然会减少冲突状态。依据这样的原则的LR分析技术称为SLR(1) 分析技术。



4.6.4、SLR(1) 技术

若LR(0)项目集规范族中有项目集 I_K :

$$I_K : \{ \dots A \rightarrow \alpha \cdot b\mu, P \rightarrow \beta. , Q \rightarrow \gamma. , \dots \}$$

则SLR(1)技术解决冲突的依据为:

对**当前输入字符a** , 如果 $a=b$ 则 $\text{action} [k, b] = \text{移进}$

对于 $a \neq b$ 如果 $a \in \text{FOLLOW}(P)$ 则 $\text{action} [k, a] = \text{用 } P \rightarrow \beta \text{ 归约}$

$a \in \text{FOLLOW}(Q)$ 则 $\text{action} [k, a] = \text{用 } Q \rightarrow \gamma \text{ 归约}$

这里面认定的前提有:

$b \notin \text{FOLLOW}(P)$ //不会产生 $P \rightarrow \beta.$, 移入b的移进/归约冲突

$b \notin \text{FOLLOW}(Q)$ //不会产生 $Q \rightarrow \gamma.$, 移入b的移进/归约冲突

$\text{FOLLOW}(P) \cap \text{FOLLOW}(Q) = \phi$ //不会产生 $P \rightarrow \beta.$ $Q \rightarrow \gamma.$ 归约/归约冲突

能用SLR(1)技术解决冲突的文法称为SLR(1)文法。该文法无二义。

一般而言，假定LR(0)规范族的一个项目集I中含有m个移进项目：

$$A_1 \rightarrow \alpha \cdot a_1 \beta_1, \quad A_2 \rightarrow \alpha \cdot a_2 \beta_2, \quad \dots, \\ A_m \rightarrow \alpha \cdot a_m \beta_m$$

同时含有n个归约项目：

$$B_1 \rightarrow \alpha \cdot, B_2 \rightarrow \alpha \cdot, \dots, B_n \rightarrow \alpha \cdot$$

如果集合 $\{a_1, \dots, a_m\}$ 、 $\text{FOLLOW}(B_1)$ 、 $\dots$ 、 $\text{FOLLOW}(B_n)$ 两两不相交(包括不得有两个FOLLOW集含有“#”)，则隐含在I中的动作冲突可通过检查现行输入符号a属于上述n+1个集合中的哪个集合而获得解决，

即：

(1) 若 a 是某个 a_i ， $i=1,2,\dots,m$ ，则移进；

(2) 若 $a \in \text{FOLLOW}(B_i)$ ， $i=1,2,\dots,n$ ，则用产生式 $B_i \rightarrow \alpha$ 进行归约；

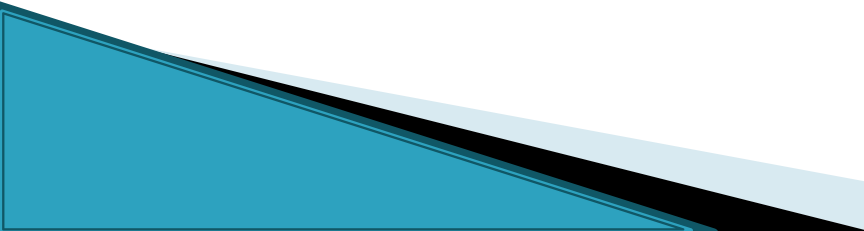
(3) 对(1)、(2)项以外的情况，报错。

冲突性动作的这种解决办法叫做SLR(1)解决办法。

对任给的一个文法 $G[S]$ ，我们可用如下办法构造它的SLR(1)分析表：先把 $G[S]$ 拓广为 $G'[S']$ ，对 $G'[S']$ 构造LR(0)项目集规范族 C 和活前缀识别自动机的状态转换函数 GO ，然后再使用 C 和 GO 按下面的算法构造 $G'[S']$ 的SLR(1)分析表。

假定 $C=\{I_0, I_1, \dots, I_n\}$ ，令每个项目集 I_k 的下标 k 为分析器的一个状态，则 $G'[S']$ 的SLR(1)分析表含有状态 $0, 1, \dots, n$ 。令那个含有项目 $S' \rightarrow \cdot S$ 的 I_k 的下标 k 为初态，则函数ACTION和GOTO可按如下方法构造：

(1) 若项目 $A \rightarrow \alpha \cdot a \beta$ 属于 I_k 且 $GO(I_k, a)=I_j$ ， a 为终结符，则置ACTION[k,a]为“将状态 j 和符号 a 移进栈”，简记为“ s_j ”。

- (2) 若项目 $A \rightarrow \alpha \cdot$ 属于 I_k ，那么对任何输入符号 a ， $a \in \text{FOLLOW}(A)$ ，置 $\text{ACTION}[k,a]$ 为“用产生式 $A \rightarrow \alpha$ 进行归约”，简记为“ r_j ”。其中， j 是产生式的编号，即 $A \rightarrow \alpha$ 是文法 $G'[S']$ 的第 j 个产生式。
- (3) 若项目 $S' \rightarrow S \cdot$ 属于 I_k ，则置 $\text{ACTION}[k,\#]$ 为“接受”，简记为“acc”。
- (4) 若 $\text{GO}(I_k, A) = I_j$ ， A 为非终结符，则置 $\text{GOTO}[k,A] = j$ 。
- (5) 分析表中凡不能用规则(1)~(4)填入信息的空白格均置上“出错标志”。
- 

按上述算法构造的含有ACTION和GOTO两部分的分析表，如果每个入口不含多重定义，则称它为文法 $G[S]$ 的一张SLR(1)表，具有SLR(1)表的文法 $G[S]$ 称为一个SLR(1)文法。数字“1”的意思是在分析过程中最多只要向前看一个符号(实际上仅是在归约时需要向前看一个符号)，使用SLR(1)表的分析器叫做一个SLR(1)分析器。

若按上述算法构造的分析表存在多重定义的入口(即含有动作冲突)，则说明文法 $G[S]$ 不是SLR(1)的；在这种情况下，不能用上述算法构造分析器。

SLR(1)的ACTION和GOTO表构造

类似LR(0)的分析表构造，假定 $C=\{I_0, I_1, \dots, I_n\}$ ，即：
 G' 的SLR分析表含有状态 $0, 1, \dots, n$ 。又因为SLR(1)文法要向前看一个字符，因此动作不仅应根据当前状态，还应根据输入缓冲区的第一个符号做出。则改一维函数ACTION(S)为二维函数ACTION(S, a)：

按上述算法构造的含有ACTION和GOTO两部分的分析表，如果每个入口不含多重定义，则称它为文法G的一张SLR表。具有SLR表的文法G称为一个SLR(1)文法。数字1的意思是，在分析过程中顶多只要向前看一个符号。

SLR(1)的ACTION和GOTO表构造

- 若项目 $A \rightarrow \alpha \cdot a \beta$ 属于 I_k 且 $GOTO(I_k, a) = I_j$, a 为终结符, 则置 $ACTION[k, a]$ 为 “把状态 j 和符号 a 移进栈”, 简记为 “ S_j ”;
- 若项目 $A \rightarrow \alpha \cdot$ 属于 I_k , 那么, 对任何输入符号 a , 如果 $a \in FOLLOW(A)$, 则置 $ACTION[k, a]$ 为 “用产生式 $A \rightarrow \alpha$ 进行规约”, 简记为 “ r_j ”; 其中, 假定 $A \rightarrow \alpha$ 为文法 G' 的第 j 个产生式;
- 若项目 $S' \rightarrow S \cdot$ 属于 I_k , 则置 $ACTION[k, \#]$ 为 “接受”, 简记为 “acc”;
- 若 $GOTO(I_k, A) = I_j$, $A \in V_N$, 则置 $GOTO(k, A) = j$;
- 分析表中凡不能用规则1至4填入信息的空白格均置上 “error”(或空置)。

- | | | | |
|----|-----------------------|----|-----------------------|
| 0) | $E' \rightarrow E$ | 3) | $T \rightarrow T * F$ |
| 1) | $E \rightarrow E + T$ | 4) | $T \rightarrow F$ |
| 2) | $E \rightarrow T$ | 5) | $F \rightarrow (E)$ |
| | | 6) | $F \rightarrow id$ |



Go(I, X)	状态I	项目核心J	计算closure(J)后添加的项目
	0	(0, 0)	(1, 0) (2, 0) (3, 0) (4, 0) (5, 0) (6, 0)
go(0, E)	1	(0, 1) (1, 1)	
go(0, T)	2	(2, 1) (3, 1)	
go(0, F)	3	(4, 1)	
go(0, ())	4	(5, 1)	(1, 0) (2, 0) (3, 0) (4, 0) (5, 0) (6, 0)
go(0, id)	5	(6, 1)	
go(1, +)	6	(1, 2)	(3, 0) (4, 0) (5, 0) (6, 0)
go(2, *)	7	(3, 2)	(5, 0) (6, 0)
go(4, E)	8	(5, 2) (1, 1)	
go(4, T)	2	(2, 1) (3, 1)	

go(4, F)	3	(4, 1)	
go(4, ())	4	(5, 1)	(1, 0) (2, 0) (3, 0) (4, 0) (5, 0) (6, 0)
go(4, id)	5	(6, 1)	
go(6, T)	9	(1, 3) (3, 1)	
go(6, F)	3	(4, 1)	
go(6, ())	4	(5, 1)	(1, 0) (2, 0) (3, 0) (4, 0) (5, 0) (6, 0)
go(6, id)	5	(6, 1)	
go(7, F)	10	(3, 3)	
go(7, ())	4	(5, 1)	(1, 0) (2, 0) (3, 0) (4, 0) (5, 0) (6, 0)
go(7, id)	5	(6, 1)	
go(8,))	11	(5, 3)	
go(8, +)	6	(1, 2)	(3, 0) (4, 0) (5, 0) (6, 0)
go(9, *)	7	(3, 2)	(5, 0) (6, 0)

至此，所有状态转移的可能都被考虑到了，不可能再产生新的状态。

$$\begin{array}{ll} 0) E' \rightarrow E & 3) T \rightarrow T * F \\ 1) E \rightarrow E + T & 4) T \rightarrow F \\ 2) E \rightarrow T & 5) F \rightarrow (E) \\ & 6) F \rightarrow \text{id} \end{array}$$
$$\begin{aligned}\text{Follow}(E') &= \{ \# \} \\ \text{Follow}(E) &= \text{Follow}(E') \cup \{ +,) \} \\ &= \{ +,), \# \} \\ \text{Follow}(T) &= \text{Follow}(E) \cup \{ * \} = \{ +,), \#, * \} \\ \text{Follow}(F) &= \text{Follow}(T) = \{ +,), \#, * \}\end{aligned}$$

0	(0, 0) (1, 0) (2, 0) (3, 0) (4, 0) (5, 0) (6, 0)
1	(0, 1) (1, 1)
2	(2, 1) (3, 1)
3	(4, 1)
4	(5, 1) (1, 0) (2, 0) (3, 0) (4, 0) (5, 0) (6, 0)
5	(6, 1)
6	(1, 2) (3, 0) (4, 0) (5, 0) (6, 0)
7	(3, 2) (5, 0) (6, 0)
8	(5, 2) (1, 1)
2	(2, 1) (3, 1)
3	(4, 1)
4	(5, 1) (1, 0) (2, 0) (3, 0) (4, 0) (5, 0) (6, 0)
5	(6, 1)
9	(1, 3) (3, 1)
3	(4, 1)
4	(5, 1) (1, 0) (2, 0) (3, 0) (4, 0) (5, 0) (6, 0)
5	(6, 1)
10	(3, 3)

	ACTION	GOTO							
		+	*	(	)	id	E	T	F
0	S			4		5	1	2	3
1	acc、S	6							
2	r2、S		7						
3	r4								
4	S			4		5	1	2	3
5	r6								
6	S			4		5		9	3
7	S			4		5			10
8	S	6			11				
9	S、r1		7						
10	r3								
11	r5								

- 0) $E' \rightarrow E$
1) $E \rightarrow E + T$
2) $E \rightarrow T$

3) $T \rightarrow T * F$
4) $T \rightarrow F$
5) $F \rightarrow (E)$
6) $F \rightarrow id$

$Follow(E') = \{ \# \}$
 $Follow(E) = Follow(E') \cup \{ +,) \}$
 $\quad = \{ +,), \# \}$
 $Follow(T) = Follow(E) \cup \{ * \} = \{ +,), \#, * \}$
 $Follow(F) = Follow(T) = \{ +,), \#, * \}$

	ACTION						GOTO		
	+	*	(	)	id	#	E	T	F
0			S ₄		S ₅		1	2	3
1	S ₆					acc			
2	r2	S ₇		r2		r2			
3	r4	r4		r4		r4			
4			S ₄		S ₅		1	2	3
5	r6	r6		r6		r6			
6			S ₄		S ₅			9	3
7			S ₄		S ₅				10
8	S ₆			S ₁₁					
9	r1	S ₇		r1		r1			
10	r3	r3		r3		r3			
11	r5	r5		r5		r5			

G[S]:

S $\rightarrow$ BB

B $\rightarrow$ aB

B $\rightarrow$ b

例3.18 试构造例3.16所示文法G[S]的SLR(1)分析表。

[解答] 构造SLR(1)分析表必须先求出所有形如“A $\rightarrow$ $\alpha \cdot$ ”的FOLLOW(A)，即对文法G'[S']的归约项目：

B $\rightarrow$ b $\cdot$

S $\rightarrow$ BB $\cdot$

B $\rightarrow$ aB $\cdot$

求FOLLOW集，由FOLLOW集的构造方法得：

$$\text{FOLLOW}(S) = \{ \# \}$$

$$\text{FOLLOW}(B) = \{ a, b, \# \}$$

根据SLR(1)分析表的构造方法得文法 $G'[S']$ 的SLR(1)分析表见表3.17。

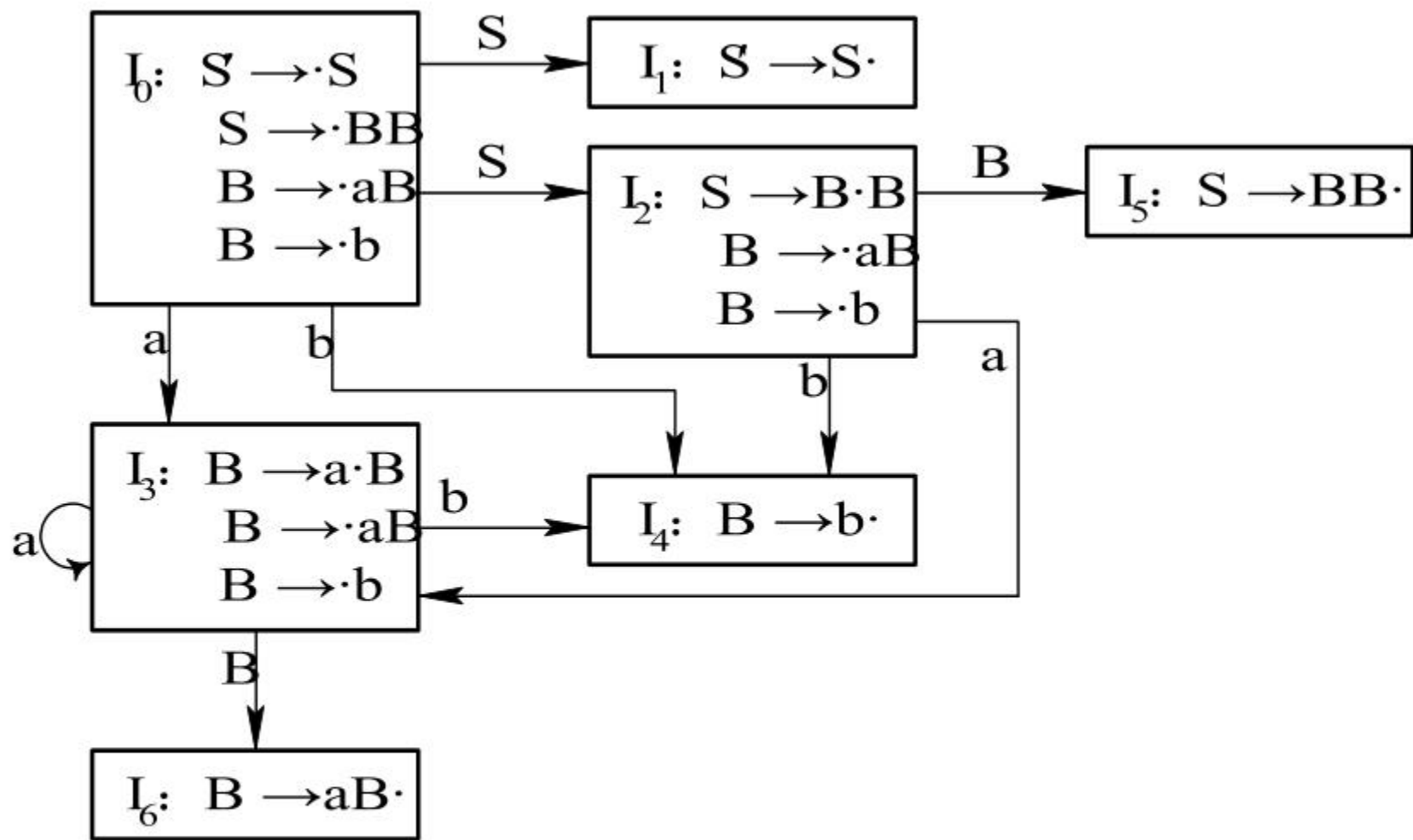


图3-22 例3.16文法 $G'[S']$ 的DFA(即LR(0)项目集和GO函数)

表3.15 例3.16的LR(0)分析表

状态	ACTION			GOTO	
	a	b	#	S	B
0	s_3	s_4		1	2
1			acc		
2	s_3	s_4			5
3	s_3	s_4			6
4	r_3	r_3	r_3		
5	r_1	r_1	r_1		
6	r_2	r_2	r_2		

表3.17 例3.18的SLR(1)分析表

状 态	ACTION			GOTO	
	a	b	#	S	B
0	s_3	s_4		1	2
1			acc		
2	s_3	s_4			5
3	s_3	s_4			6
4	r_3	r_3	r_3		
5			r_1		
6	r_2	r_2	r_2		

练习：文法G[S]

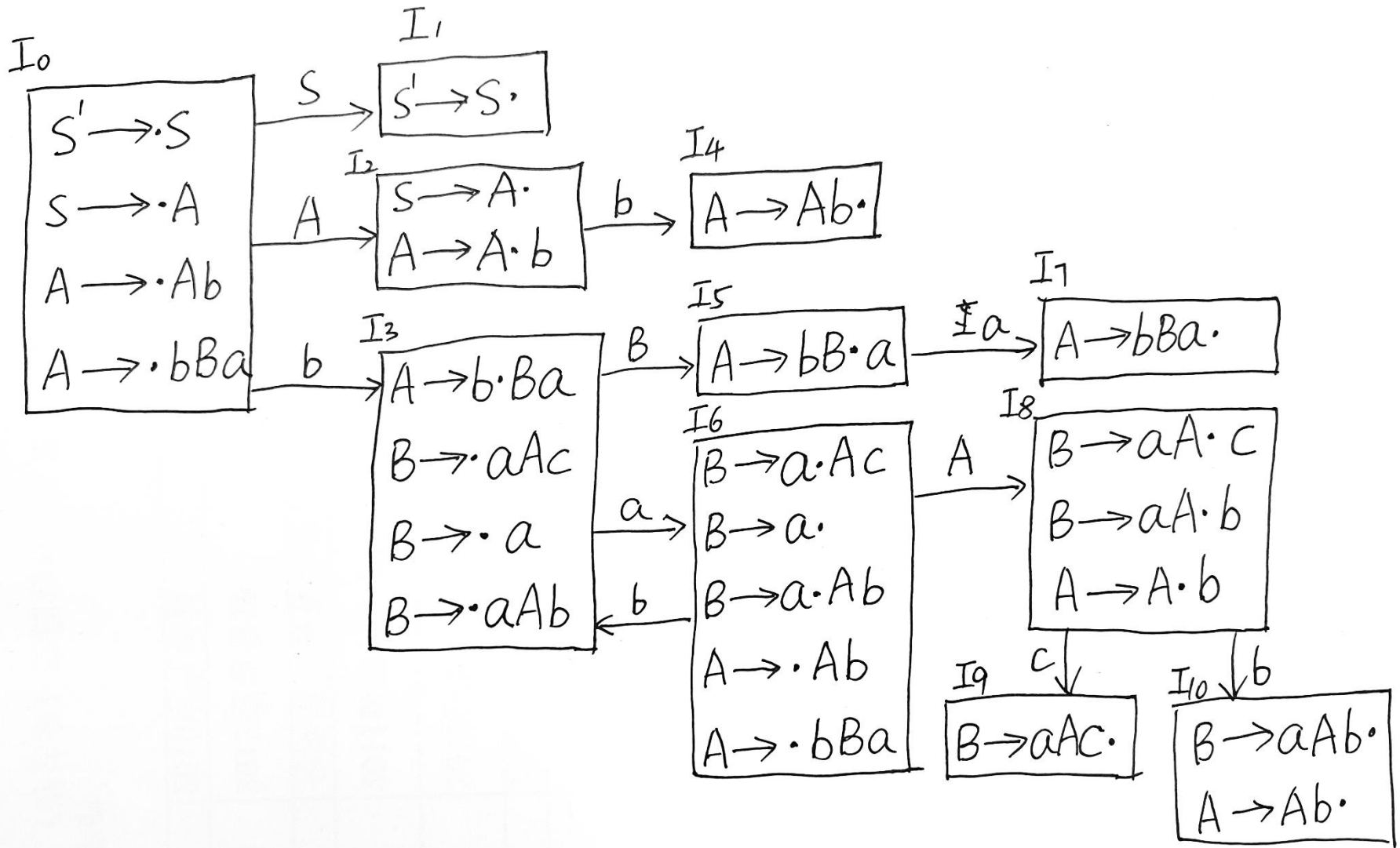
$S \rightarrow A$

$A \rightarrow Ab \mid bBa$

$B \rightarrow aAc \mid a \mid aAb$

- (1) 构造SLR(1)分析表（ACTION及GOTO表）
- (2) 该文法是否SLR(1)文法？

SLR(1)文法对句子的分析过程与LR(0)一样，试用其分析句子**babaacab**



		Action				Goto		
		a	b	c	#	S	A	B
S	0		3			1	2	
acc	1							
S, r1	2		4					
S	3	6						5
r2	4							
S	5	7						
S, r5	6		3				8	
r3	7							
S	8		10	9				
r4	9							
r2, r6	10							

$$\text{Follow}(S) = \{\#\}$$

$$\text{Follow}(A) = \{\#, b, c\}$$

$$\text{Follow}(B) = \{a\}$$

	Action				Goto		
	a	b	c	#	S	A	B
0		S3			1	2	
1				acc			
2		S4					
3	S6						5
4							
5	S7						
6		S3				8	
7							
8		S10	S9				
9							
10							

$\text{Follow}(S) = \{\#\}$

$\text{Follow}(A) = \{\#, b, c\}$

$\text{Follow}(B) = \{a\}$

	Action				Goto		
	a	b	c	#	S	A	B
0		S3			1	2	
1				acc			
2		S4		r1			
3	S6						5
4		r2	r2	r2			
5	S7						
6	r5	S3				8	
7		r3	r3	r3			
8		S10	S9				
9	r4						
10	r6	r2	r2	r2			

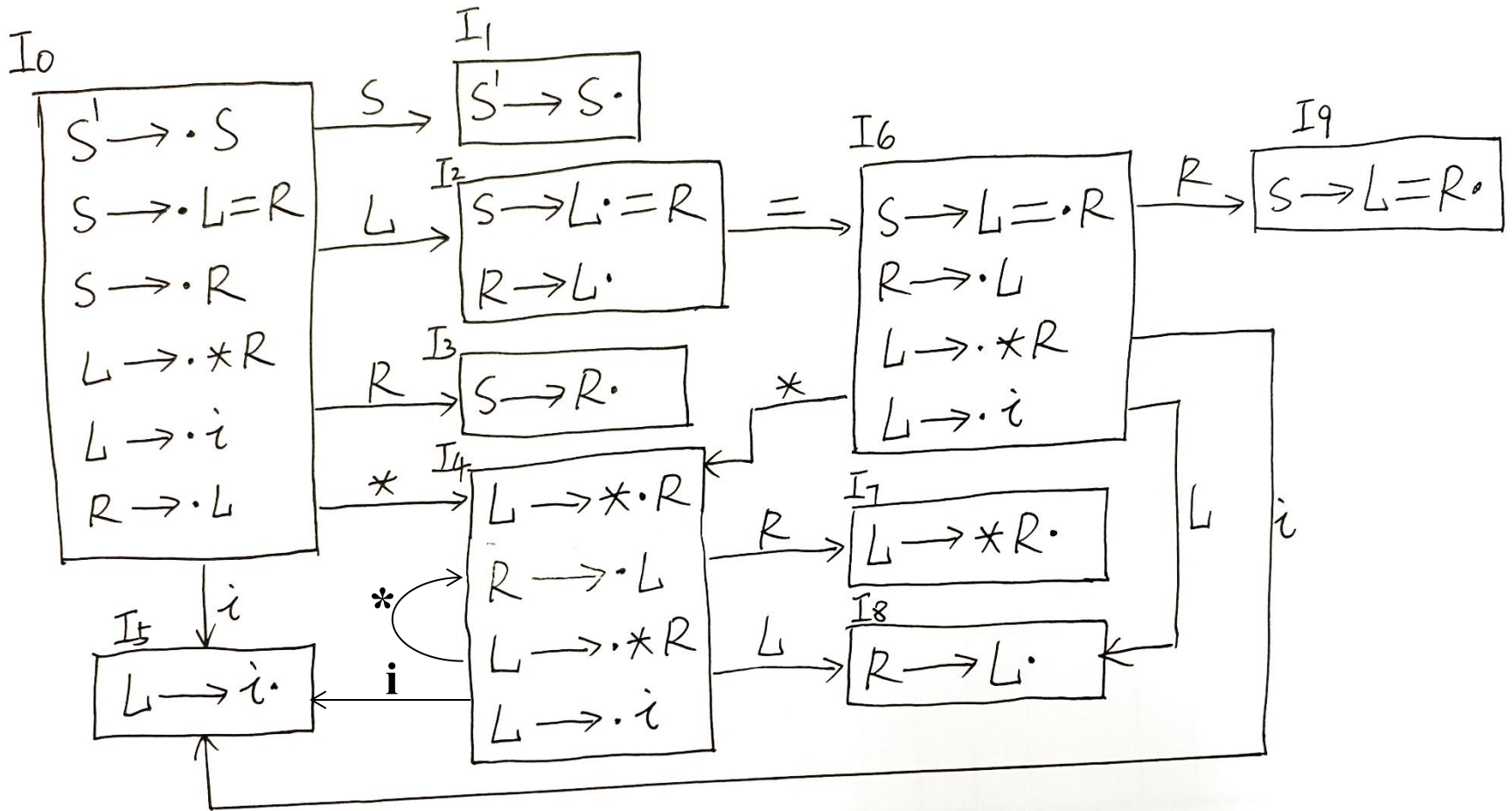
LR(0)分析算法

- ▶ 置ip指向输入串w的第一个符号，初始状态入状态栈，#入符号栈
 - 令St为栈顶状态，a是ip指向的符号
 - LOOP
 - begin
 - if $S \in \text{ACTION}[St]$
 - then begin PUSH GOTO(St, a), a(进栈——状态栈和符号栈)
ip 前进(指向下一输入符号)
 - end
 - else if $r_j \in \text{ACTION}[St]$ (第j条产生式为 $A \rightarrow \beta$)
 - then begin 两个栈各 pop $|\beta|$ 项
假设当前状态栈顶状态为 St' ，则
push GOTO[St' , A]和A
 - end
 - else if $acc \in \text{ACTION}[St]$
 - then return (成功)
 - else error
 - end. LOOP

扩展文法G[S]: (0) $S' \rightarrow S$ (1) $S \rightarrow L=R$ (2) $S \rightarrow R$
 (3) $L \rightarrow *R$ (4) $L \rightarrow i$ (5) $R \rightarrow L$

构造LR(0)项目集规范族和GOTO函数

	状态	核心	闭包
	0	(0, 0)	(1, 0), (2, 0), (3, 0), (4, 0), (5, 0)
GOTO(0, S)	1	(0, 1)	
GOTO(0, L)	2	(1, 1), (5, 1)	
GOTO(0, R)	3	(2, 1)	
GOTO(0, *)	4	(3, 1)	(5, 0), (3, 0), (4, 0)
GOTO(0, id)	5	(4, 1)	
GOTO(2, =)	6	(1, 2)	(5, 0), (3, 0), (4, 0)
GOTO(4, R)	7	(3, 2)	
GOTO(4, L)	8	(5, 1)	
GOTO(6, R)	9	(1, 3)	
GOTO(6, L)	8	(5, 1)	
GOTO(6, *)	4	(3, 1)	(5, 0), (3, 0), (4, 0)
GOTO(6, i)	5	(4, 1)	



则该文法的LR(0)项目集规范族为:

I_0 : $S' \rightarrow \cdot S$ $S \rightarrow \cdot L=R$ $S \rightarrow \cdot R$ $L \rightarrow \cdot *R$ $L \rightarrow \cdot i$ $R \rightarrow \cdot L$

I_1 : $S' \rightarrow S \cdot$

I_2 : $S \rightarrow L \cdot =R$
 $R \rightarrow L \cdot$

I_3 : $S \rightarrow R \cdot$

I_4 : $L \rightarrow \cdot *R$
 $R \rightarrow \cdot L$
 $L \rightarrow \cdot i$

I_5 : $L \rightarrow i \cdot$

I_6 : $S \rightarrow L \cdot =R$
 $R \rightarrow \cdot L$
 $L \rightarrow \cdot *R$
 $L \rightarrow \cdot i$

I_7 : $L \rightarrow *R \cdot$

I_8 : $R \rightarrow L \cdot$

I_9 : $S \rightarrow L=R \cdot$

结论: 不是LR(0)文法

$\because$ I_2 项目集中, $S \rightarrow L \cdot =R$ $R \rightarrow L \cdot$ 存在移进/归约冲突

SLR能否解决 I_2 中的冲突

∵ $\text{Follow}(R) = \{\#, =\}$ 与 $\text{Follow}(L) = \{=\}$ 的交不为空，所以也不是SLR(1)文法

I_2 状态的 $S \rightarrow L. =R$ $R \rightarrow L.$ ，当下一输入符为‘=’时，若用 $R \rightarrow L$ 归约则形成“ $R=...$ ”，而观察文法得知“ $R=$ ”不是活前缀。

不同LR方法之间的主要区别就在于归约的判定方法上：

- ✓ LR(0)不展望当前输入

- ✓ 在需要用产生式 $B \rightarrow \gamma$ 进行归约时，SLR(1)方法展望当前输入，但只接受 $\text{Follow}(B)$ 中的元素。即，无论是归约为产生式右部任何位置上的 B ，都使用统一的展望符集。

- ✓ LR(1)也展望当前输入，但对不同位置上的 B 进行归约时，采用不同的展望符集，因此可大大减少移进/归约冲突。

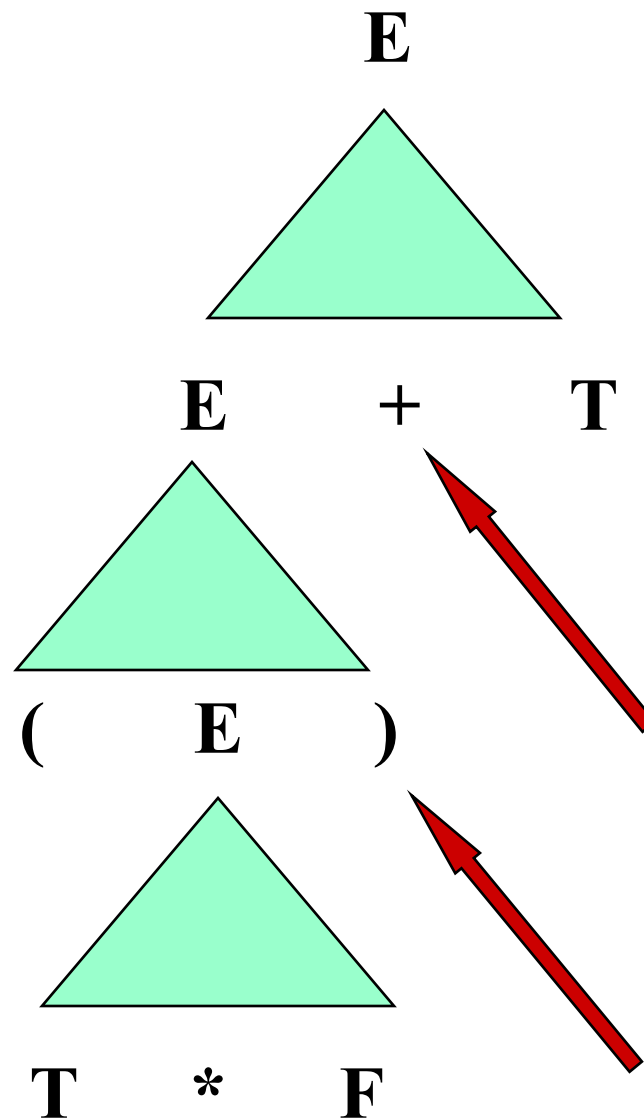
假设有文法:

$$\begin{aligned}Z &\rightarrow B^1 a B^2 c \\Z &\rightarrow B^3 d \\B &\rightarrow b\end{aligned}$$

很显然归约为 B^1 的展望符只能是 a ，归约为 B^2 的展望符只能是 c ，归约为 B^3 的展望符只能是 d 。如果用SLR(1)分析，则 $\text{Follow}(B) = \{ a, c, d \}$ ，当 B^2 的后继符为 a 或 d 时也要进行归约。

- 不同的归约中有不同的后继符。

- 特定位置的后继符是FOLLOW集的子集



4.6.5、二义性文法的处理

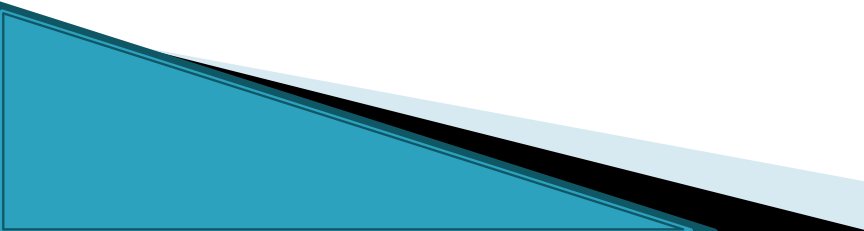
- 各种语法分析器的生成器都拒绝接受二义性文法，因为它将引起语法分析动作的不确定性。但对其利用得好则分析器效率会较高，因为二义性文法较简单。
- 但**必须去掉冲突**情况，因为任何二义文法决不是一个LR文法。一般方法是**规定运算符的优先关系和结合规则**。如表达式的二义文法可定义 $*$ 优先于 $+$ ，并采用左结合规则。体现在 $E \rightarrow E + E \cdot$ 的展望符为 $*$ 时要做移进动作。而对于**归约-归约冲突**的处理则可规定**产生式的优先顺序**
- **没有一种有效的冲突消除法。**

4.7、语法错误处理

- ✓ 发现一个错误就停止分析；
- ✓ 力图继续发现错误。

好的语法分析器应采用后一种方法。继续进行会产生伪错误：瀑布式的。错误处理不是错误改正，一般来说，把错误程序改为正确的几乎不可能，但只要能指出便达到目的。

错误处理的两种方法：

- 错误恢复：目的是指出错误，继续分析。
 - 错误修复：试图把错误程序修复为合法程序。
- 

3.5.7 LR分析器应用拓展

LR分析器除了应用于编译程序外，还可拓展到其它领域，如人工智能领域。

人工智能中采用的搜索方法如深度优先法、广度优先法、爬山法和 $\alpha - \beta$ 法等都可适用于一般的智能推测。由于要综合考虑各种因素，且推测的目标并不事先确定，故这些方法实质上是一种试探法或穷举法。因此，它们的共同特点是适用面广，但效率不高。

LR分析器的推理过程是依据“历史”来展望“未来”，因此LR分析器同样具有智能性，但是这种智能却是有限的。作为归约过程的“历史”材料的积累虽不困难(已保存于栈中)，但是“展望”材料的汇集却很不容易。因为根据历史推测未来，即使是推测未来的一个符号，也常常存在着非常多的不同可能性，所以在把“历史”和“展望”材料综合在一起时，复杂性就大为增加了。



因此，具体实现LR分析器的功能时通常采用某种限制性措施，尽可能使出现的状态减少，只有这样才能提高效率并易于实现。如简单LR——极有使用价值的SLR(1)方法，它只在栈中保留已扫描过的那段输入符号的部分信息，即“历史”，并根据这些信息和未来的一串符号决定下一步的操作。SLR(1)的这种做法使得状态数大为减少，因此可以高效率地实现。



我们可以将上述方法和思想运用于人工智能领域。通过分析可以发现，人工智能的推理按目标可分为两类：一类是对未知目标的推理；一类是对已知目标的推理。对事先无法知道的目标的推理一般采用常规的智能搜索方法——试探法或穷举法；而对已知目标的推理，虽然仍可采用试探法或穷举法，但能否采用其它更加简单有效的方法呢？LR分析器中的“历史”类同于人工智能中已搜索过的路径，而LR分析器对未来的“展望”恰好就是对已知目标的“展望”。



LR分析器是在“目标已知”这个特定条件下对人工智能常规搜索方法的一种简化，由于LR分析器推理的目标确定，故其效率远远高于常规的人工智能搜索方法。对那些事先可以分析出目标的推理问题，如智能化教学系统、智能化管理系统以及智能控制机床等，都可以采用LR分析器方法。LR分析器是一种基于目标的智能推理方法，它适用于目标确定的智能化推理问题。



作业：该文法是否LR(0)文法？是否SLR(1)文法？如果是，请给出分析表。

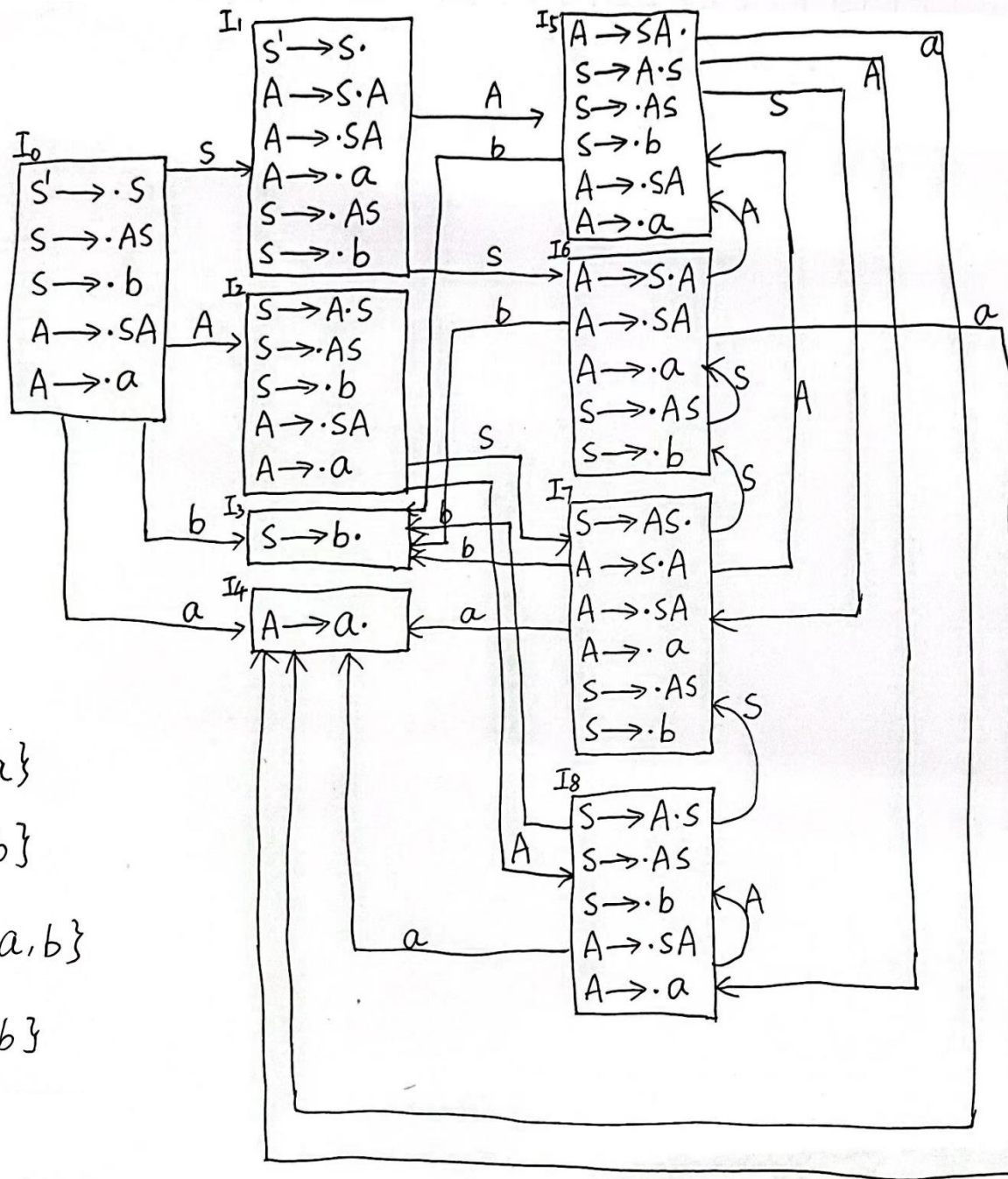
$$S \rightarrow AaAb \mid BbBa$$
$$A \rightarrow \epsilon$$
$$B \rightarrow \epsilon$$

选做：文法G[S]

$$S \rightarrow AS \mid b$$
$$A \rightarrow SA \mid a$$

该文法是否LR(0)文法？是否SLR(1)文法？给出判断步骤。





$\text{First}(S) = \{b, a\}$

$\text{First}(A) = \{a, b\}$

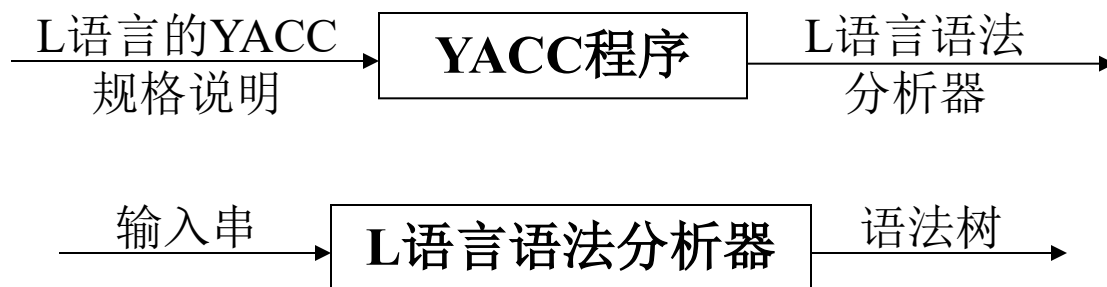
$\text{Follow}(S) = \{\#, a, b\}$

$\text{Follow}(A) = \{a, b\}$

4.7、语法分析器自动生成工具YACC

YACC(Yet Another Compiler - Compiler)是由**S.C.Johnson**等人在**AT&T**贝尔实验室研制开发的，早期作为**UNIX**操作系统中的一个实用程序。

YACC不能产生完整的编译程序。它输入语言的语法描述规格说明，基于**LALR**语法分析原理，自动构造该语言的语法分析器，同时还产生语义动作的翻译程序。



YACC规格说明：说明部分、翻译规则和辅助过程三部分

例：台式计算器的翻译程序

```
%{ #include <ctype.h> %}
```

```
%token DIGIT //DIGIT是token类型的词汇，后面可引用
```

```
%%
```

```
line      :      expr'\n'          {printf("%d\n", $1);}
```

```
;
```

```
expr      :      expr'+'term      { $$ = $1 + $3; }
```

```
|
```

```
term
```

```
;
```

```
term      :      term'*'factor    { $$ = $1 * $3; }
```

```
|
```

```
factor
```

```
;
```

```
factor    :      '(' expr ')'      { $$ = $2; }
```

```
|
```

```
DIGIT
```

```
;
```

```
%%
```

```
yylex() //辅助过程，如词法部分、错误处理部分等
```

```
{ int c; c = getchar( );
```

```
  if(isdigit(c))
```

```
  { yylval = c - '0'; return DIGIT; }
```

```
  return c;
```

```
}
```

产生式右部第一个符号的值

产生式左部符号的值